NAME: VINAY BHARGAV REDDY ID NO:2400031280 SEC NO : 12

FULL STACK DEVLOPMENT

Skill-2: Hibernate CRUD (Students, MySQL)

Purpose: CRUD on products table using JPA annotations and Hibernate.
Tech stack: Java 21, Maven, Hibernate 6.4.x, MySQL Connector/J, SLF4J Simple.

Project files (full contents)

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>skill-2</artifactId>
  <version>1.0-SNAPSHOT</version>
  <name>skill-2</name>
  <description>Skill 2: Hibernate CRUD for Product inventory</description>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <hibernate.version>6.4.8.Final</hibernate.version>
    <mysql.connector.version>8.0.33</mysql.connector.version>
    <jakarta.persistence.version>3.1.0</jakarta.persistence.version>
    <jakarta.transaction.version>2.0.1</jakarta.transaction.version>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
```

2400031280

```
        <version>${hibernate.version}</version>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>${mysql.connector.version}</version>
    </dependency>
    <dependency>
        <groupId>jakarta.persistence</groupId>
        <artifactId>jakarta.persistence-api</artifactId>
        <version>${jakarta.persistence.version}</version>
    </dependency>
    <dependency>
        <groupId>jakarta.transaction</groupId>
        <artifactId>jakarta.transaction-api</artifactId>
        <version>${jakarta.transaction.version}</version>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>2.0.16</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <source>${maven.compiler.source}</source>
                <target>${maven.compiler.target}</target>
                <release>${maven.compiler.release}</release>
```

2400031280

```
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.5.0</version>
            <configuration>
                <mainClass>com.example.App</mainClass>
            </configuration>
        </plugin>
    </plugins>
  </build>
</project>
```

Student.java

```
package com.klef;

import jakarta.persistence.*;

@Entity
@Table(name="students")
public class Student {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;

private String name;
private String course;
private int age;

public Student(){}
```

240031280

```
public Student(String name,String course,int age){
this.name=name;
this.course=course;
this.age=age;
}

public int getId(){
return id;
}

public String getName(){
return name;
}

public void setName(String name){
this.name=name;
}

public String getCourse(){
return course;
}

public void setCourse(String course){
this.course=course;
}

public int getAge(){
return age;
}

public void setAge(int age){
this.age=age;
}
```

2400031280

```
@Override
public String toString(){
return "Student{id="+id+", name="+name+", course="+course+", age="+age+"}";
}

}
```

App.java

```
package com.example;

import java.math.BigDecimal;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

/**
 * Demonstrates CRUD for Product using Hibernate.
 */
public class App {

    private static final String DEFAULT_URL = "jdbc:mysql://localhost:3306/skill2db";
    private static final String DEFAULT_USER = "hareesh";
    private static final String DEFAULT_PASSWORD = "Hari@9126";

    private static final SessionFactory sessionFactory = buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        // Allow overrides via environment variables so credentials/host changes do not require code edits.
        String url = System.getenv().getOrDefault("DB_URL", DEFAULT_URL);
        String user = System.getenv().getOrDefault("DB_USER", DEFAULT_USER);
        String password = System.getenv().getOrDefault("DB_PASSWORD", DEFAULT_PASSWORD);
```

240031280

```
    Configuration configuration = new Configuration()
            .configure("hibernate.cfg.xml")
            .setProperty("hibernate.connection.url", url)
            .setProperty("hibernate.connection.username", user)
            .setProperty("hibernate.connection.password", password)
            .addAnnotatedClass(Product.class);

    return configuration.buildSessionFactory();
}

public static void main(String[] args) {
    try {
        resetTable();
        List<Long> productIds = createProducts();

        if (!productIds.isEmpty()) {
            Long firstId = productIds.get(0);
            Long secondId = productIds.size() > 1 ? productIds.get(1) : firstId;
            Long thirdId = productIds.size() > 2 ? productIds.get(2) : firstId;

            readProduct(firstId);
            updateProduct(secondId, new BigDecimal("1199.99"), 42);
            deleteProduct(thirdId);

            listAllProducts();
        }
    } finally {
        sessionFactory.close();
    }
}

private static List<Long> createProducts() {
    Product laptop = new Product("Laptop", "15 inch business laptop", new BigDecimal("1299.99"), 10);
```

2400031280

```
    Product mouse = new Product("Mouse", "Wireless ergonomic mouse", new BigDecimal("39.99"), 75);
    Product chair = new Product("Chair", "Ergonomic office chair", new BigDecimal("249.00"), 15);

    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        session.persist(laptop);
        session.persist(mouse);
        session.persist(chair);
        session.flush(); // ensures IDs are generated
        tx.commit();
    }

    System.out.println("Inserted products: " + laptop + " | " + mouse + " | " + chair);
    return List.of(laptop.getId(), mouse.getId(), chair.getId());
}

private static void resetTable() {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        session.createMutationQuery("delete from Product").executeUpdate();
        tx.commit();
    }
}

private static void readProduct(Long id) {
    try (Session session = sessionFactory.openSession()) {
        Product found = session.get(Product.class, id);
        if (found != null) {
            System.out.println("Retrieved: " + found);
        } else {
            System.out.println("Product with id=" + id + " not found");
        }
    }
}
```

2400031280

```
private static void updateProduct(Long id, BigDecimal newPrice, int newQuantity) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        Product found = session.get(Product.class, id);
        if (found != null) {
            found.setPrice(newPrice);
            found.setQuantity(newQuantity);
            session.merge(found);
            tx.commit();
            System.out.println("Updated: " + found);
        } else {
            System.out.println("Product with id=" + id + " not found for update");
            tx.rollback();
        }
    }
}

private static void deleteProduct(Long id) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        Product found = session.get(Product.class, id);
        if (found != null) {
            session.remove(found);
            tx.commit();
            System.out.println("Deleted product id=" + id);
        } else {
            System.out.println("Product with id=" + id + " not found for delete");
            tx.rollback();
        }
    }
}

private static void listAllProducts() {
```

240031280

```
        try (Session session = sessionFactory.openSession()) {
            List<Product> products = session.createQuery("from Product", Product.class).getResultList();
            System.out.println("All products: " + products);
        }
    }
}
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 5.4//EN"
    "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/skill2db</property>
    <property name="hibernate.connection.username">hareesh</property>
    <property name="hibernate.connection.password">Hari@9126</property>

    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.current_session_context_class">thread</property>

    <mapping class="com.example.Product" />
  </session-factory>
</hibernate-configuration>
```

Build

From skill-2:

```
mvn -q compile
```

Run (with Maven [INFO] logs)

From skill-2:

```
mvn "-Dexec.mainClass=com.example.App" org.codehaus.mojo:exec-maven-plugin:3.1.0:java
```

2400031280

Expected console flow

- Deletes existing rows.
- Inserts 3 products.
- Reads first, updates second (price/quantity), deletes third.
- Prints remaining rows.
- Ends with [INFO] BUILD SUCCESS.

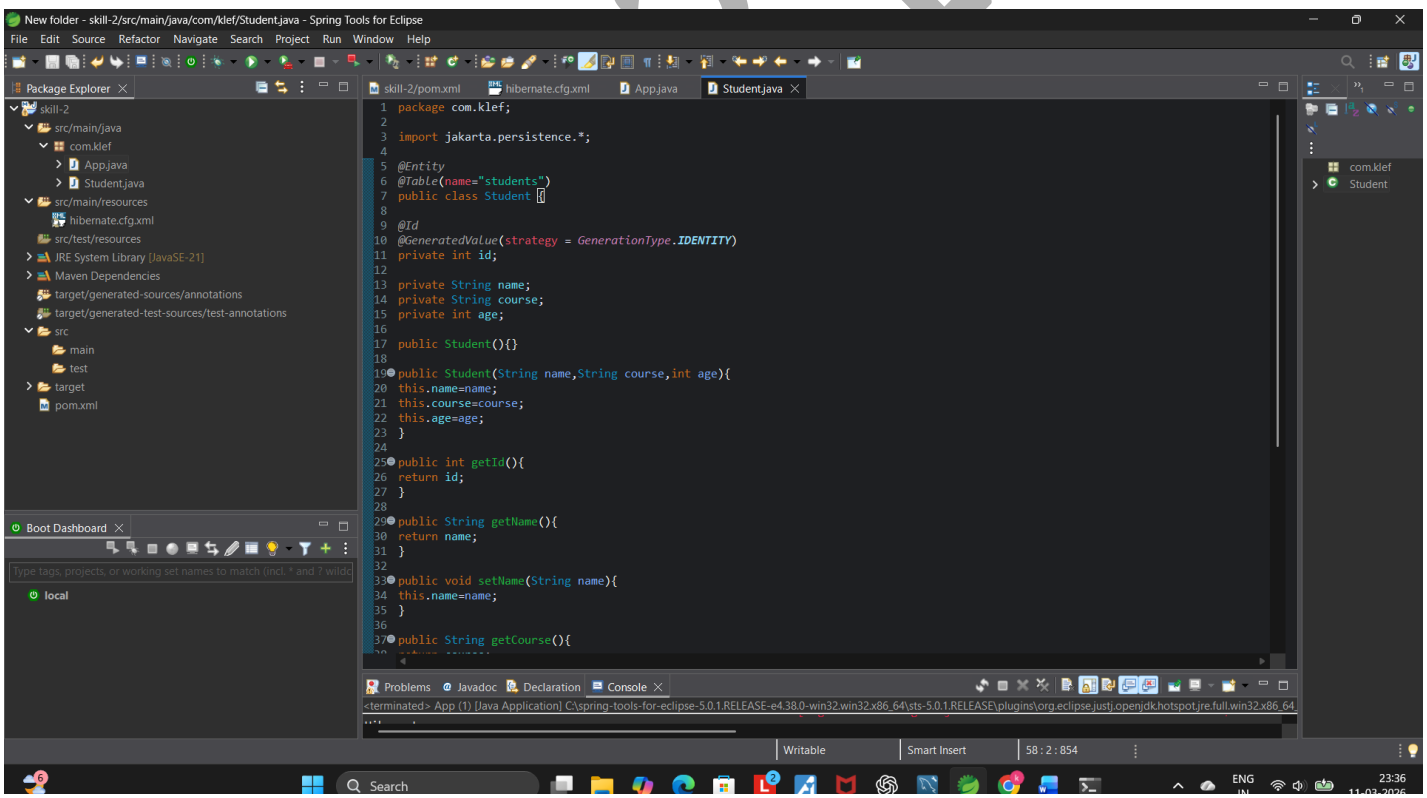
DB inspection (MySQL CLI)

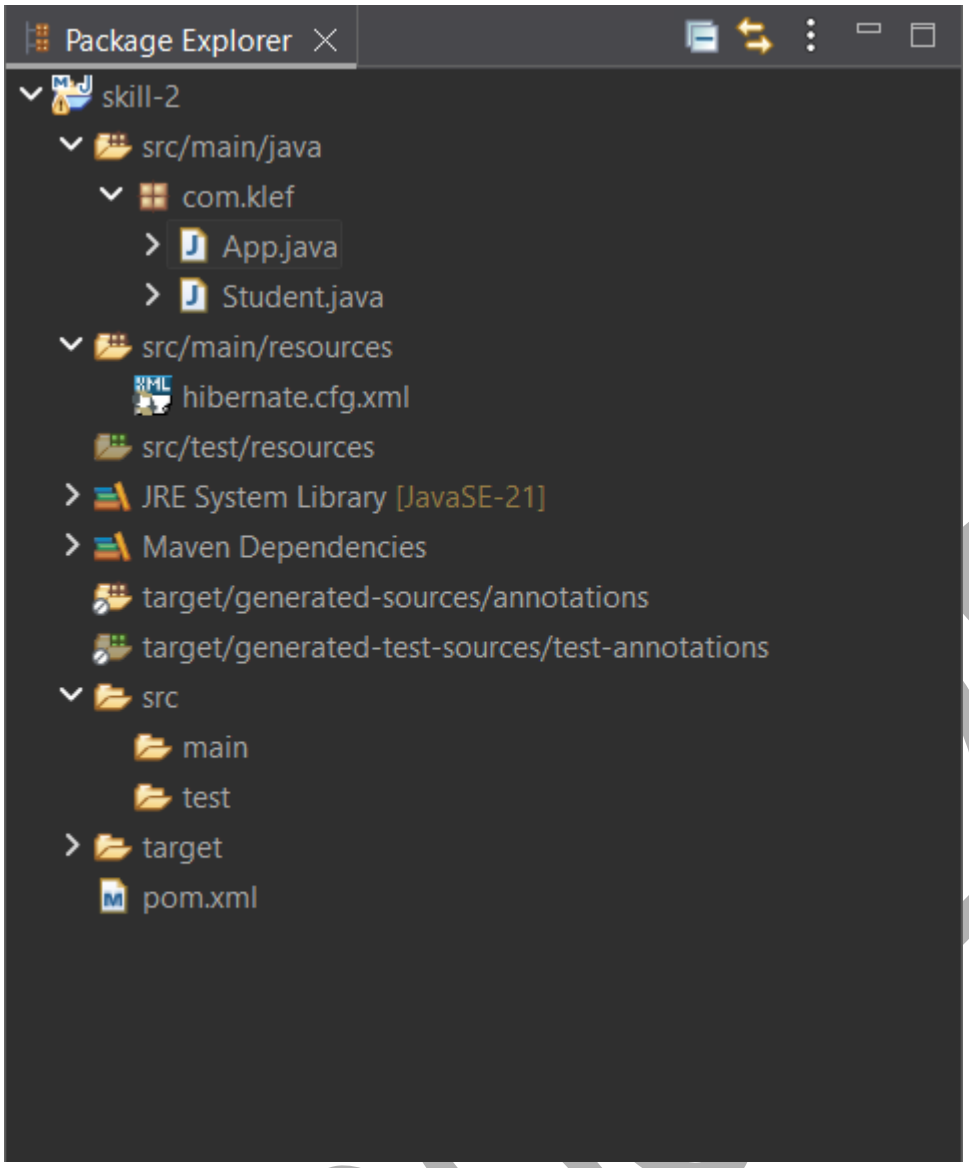
mysql -u hareesh -pHari@9126 -e "SHOW DATABASES; USE skill2db; SHOW TABLES; DESCRIBE products; SELECT * FROM products;"

What was done

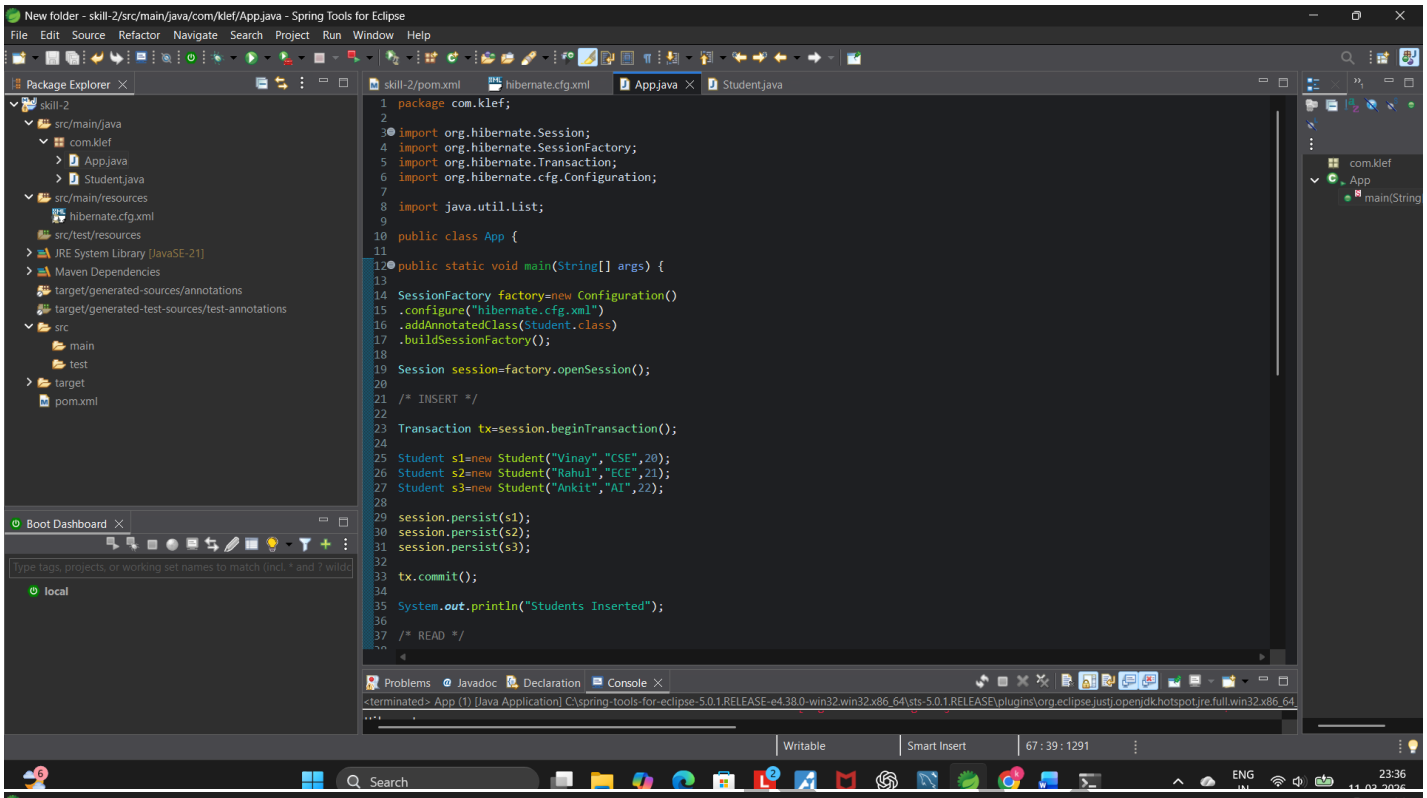
- Maven project with Hibernate/JPA and MySQL driver.
- Product entity using IDENTITY strategy.
- App performs full CRUD and prints results; resets table each run.
- Env-var overrides for DB connection details (DB_URL, DB_USER, DB_PASSWORD).

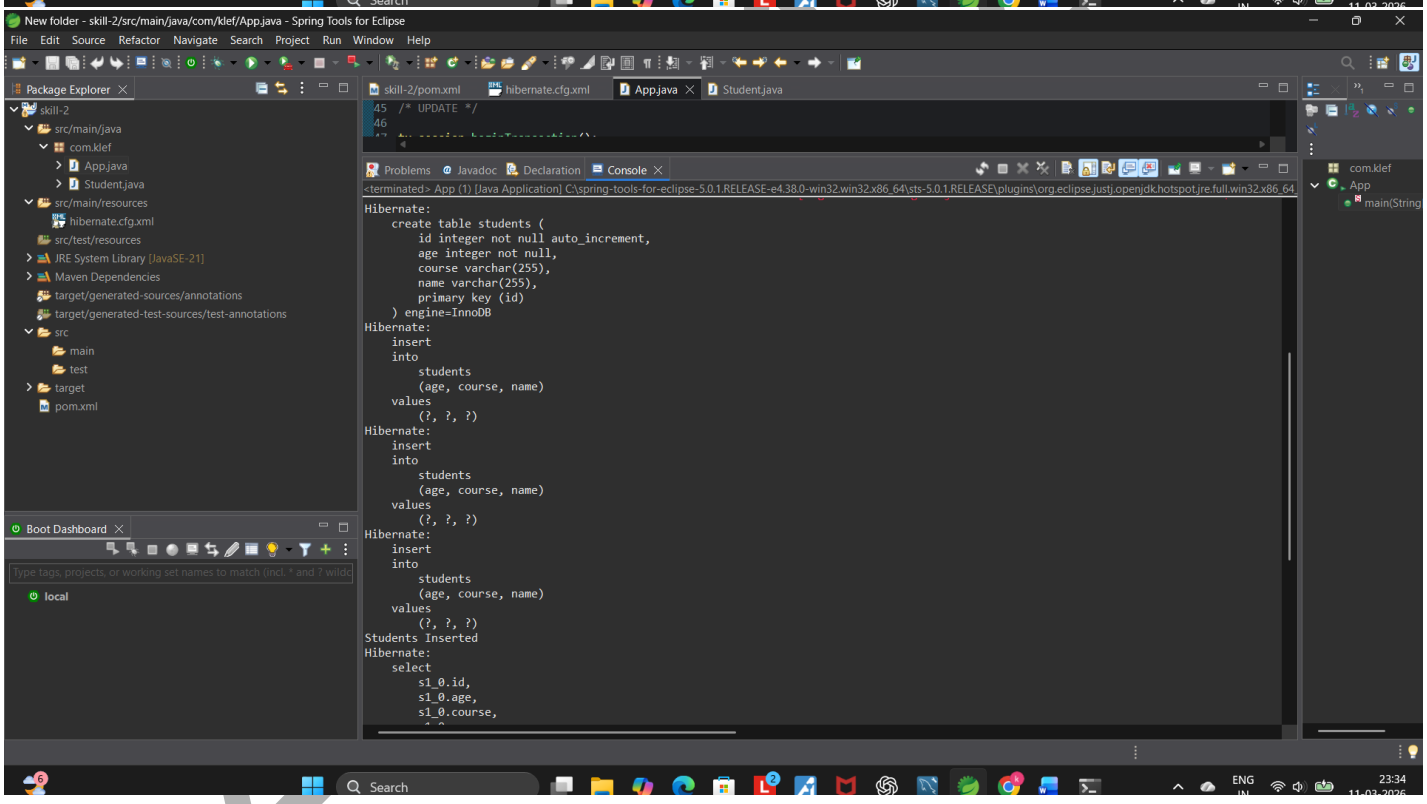
Outputs




Package Explorer
skill-2
src/main/java
com.klef
App.java
Student.java
src/main/resources
hibernate.cfg.xml
src/test/resources
JRE System Library [JavaSE-21]
Maven Dependencies
target/generated-sources/annotations
target/generated-test-sources/test-annotations
src
main
test
target
pom.xml

2400280





GitHub Repository Link

The complete source code for this experiment is available in the following GitHub repository:

Repository:
https://github.com/Kishtipati-Vinay-Bhargav-Reddy/2400031280_FSAD_SKILL_2.git